Práctica Número 1

Historia, evolución y características de Lenguajes de Programación

Objetivo: Conocer la evolución de los lenguajes de programación y sus características.

Ejercicio 1: Los lenguajes de programación más representativos son los que están abajo. Indique para cada uno de los períodos presentados cuales son las características nuevas que se incorporan y cuál de ellos la incorpora.

- **1951 - 1955**: Lenguajes tipo Assembly → Programación de bajo nivel, dependiente del hardware.
- **1956 - 1960**: FORTRAN, ALGOL 58, ALGOL 60, LISP → Lenguajes de alto nivel, primeros compiladores, programación científica (Fortran), listas y recursión (LISP).
- **1961 - 1965**: COBOL, ALGOL 60, SNOBOL, JOVIAL → Orientación a negocios (COBOL), mejoras en estructuras de control, manejo de strings (Snobol).
- **1966 - 1970**: APL, FORTRAN 66, BASIC, PL/I, SIMULA 67, ALGOL-W → Lenguajes más simples, multipropósito (PL/I), nace la orientación a objetos (Simula 67).
- **1971 - 1975**: Pascal, C, Scheme, Prolog → Programación estructurada (Pascal), bajo nivel + eficiencia (C), programación lógica (Prolog).
- **1976 - 1980**: Smalltalk, Ada, FORTRAN 77, ML → Programación orientada a objetos más consolidada (Smalltalk), tipado fuerte (Ada, ML).
- **1981 - 1985**: Smalltalk 80, Turbo Pascal, Postscript → Popularización de la POO, herramientas más amigables.
- **1986 - 1990**: FORTRAN 90, C++, SML → Más eficiencia de la POO, mejoras en lenguajes científicos.
- **1991 - 1995**: TCL, PERL, HTML → Nacimiento de la web (html), Scripts y automatización (PERL).
- **1996 - 2000**: Java, Javascript, XML → Portabilidad (Java), Web dinámica (JavaScript), estructura de datos (XML).

Ejercicio 2: Escriba brevemente la historia del lenguaje de programación que eligió en la encuesta u otro de su preferencia.

- **Python:** fue creado en 1991 por Guido van Rossum. Surgió como un lenguaje simple y fácil de leer, con una sintaxis clara que prioriza la productividad del programador. Combina varios

paradigmas (orientado a objetos, imperativo y funcional). Con el tiempo se volvió muy popular en áreas como desarrollo web, automatización, ciencia de datos e inteligencia artificial gracias a su simplicidad y gran cantidad de librerías.

- **Java:** fue desarrollado en 1995 por James Gosling en la empresa Sun Microsystems. Su principal objetivo fue crear un lenguaje portátil, capaz de ejecutarse en cualquier sistema mediante la máquina virtual de Java (JVM). Está basado en programación orientada a objetos y se hizo muy popular en aplicaciones empresariales, desarrollo web y aplicaciones móviles (especialmente Android). Su lema es “write once, run anywhere”.

Ejercicio 3: ¿Qué atributos debería tener un buen lenguaje de programación? Por ejemplo, ortogonalidad, expresividad, legibilidad, simplicidad, etc. De al menos un ejemplo de un lenguaje que cumple con las características citadas.

- Simplicidad y Legibilidad: programas fáciles de escribir y leer, lenguaje fácil de aprender y enseñar, sintaxis clara, pocas reglas, y sin ambigüedades.
- Claridad de los Bindings: asociación entre un nombre y una entidad del programa (ejemplo nombre de una variable y su valor).
- Confiabilidad: que el lenguaje permita la creación de programas seguros (lenguaje confiable evita errores comunes, tipado fuerte, detecta errores, previene problemas) Chequeo de tipos y manejo de excepciones.
- Soporte: facilita el aprendizaje, el desarrollo y el matenimiento (bibliotecas, compiladores, frameworks, intérpretes, etc.).
- Abstracción: representación de conceptos complejos en estructuras más simples, reutilización de código (abstracción de datos: clases; abstracción de procesos: funciones, módulos, interfaces).
- Ortogonalidad: las características del lenguaje funcionan de manera independiente y consistente entre sí (por ejemplo, que no pueda retornar tal tipo). Reduce excepciones y complejidad en el lenguaje.
- Eficiencia: uso eficiente de los recursos computacionales (TE, uso de memoria, consumo de CPU).

Por ejemplos Java y C# cumple con la mayoría de estas características en mayor o menor medida.

Ejercicio 4: Tome uno o dos lenguajes de los que conozca (J A V A) y:

- Describa los tipos de expresiones que se pueden escribir en él/ellos:

  Aritméticas: +, -, *, /, %

  Relacionales: ==, !=, <, >, <=, >=

  Lógicas: &&, ||, !

  De asignación: =, +=, -=

  Condicionales (ternaria): condición ? valor1 : valor2

  Expresiones con objetos: creación (new), acceso a métodos y atributos

- Describa las facilidades provistas para la organización del programa:

  Clases y objetos (programación orientada a objetos).

  Paquetes (packages) para agrupar clases.

  Métodos para dividir funcionalidades.

  Interfaces y herencia para reutilización y diseño modular.

  Encapsulamiento (modificadores como private, public).

- Indique cuáles de los atributos del ejercicio anterior posee el lenguaje elegido y cuáles no posee, justifique en cada caso.

  Simplicidad y Legibilidad: ✓ (sintaxis clara y estructurada, aunque más verbosa que otros lenguajes).

  Claridad de los Bindings: ✓ (variables deben declararse con tipo, lo que hace clara la relación nombre–tipo–valor).

  Confiabilidad: ✓ (tiene tipado fuerte, chequeo de tipos en compilación y manejo de excepciones).

  Soporte: ✓ (gran cantidad de bibliotecas, frameworks y herramientas).

  Abstracción: ✓ (permite usar clases, interfaces, métodos y herencia para modelar problemas complejos).

  Ortogonalidad: ⚠ Parcial: es bastante consistente, pero tiene algunas excepciones (por ejemplo, tipos primitivos vs objetos).

  Eficiencia: ✓ (buena eficiencia, aunque menor que C/C++ porque usa máquina virtual JVM).

Lenguaje ADA

Ejercicio 5: Describa las características más relevantes de Ada, referida a:

- Tipos de datos → Ada tiene **tipado fuerte y estricto**, lo que mejora la seguridad. Permite definir tipos propios (tipos derivados), rangos, enumeraciones y tipos escalares. Evita conversiones implícitas peligrosas.
- Tipos abstractos de datos – paquetes → Los **packages** permiten encapsular datos y operaciones. Separan **interfaz** de **implementación**, favoreciendo la modularidad y reutilización.
- Estructuras de datos → Soporta arrays, registros y tipos dinámicos. Permite crear estructuras complejas definidas por el usuario de forma segura.
- Manejo de excepciones → Incluye manejo de excepciones incorporado (exception, when). Permite capturar y manejar errores en tiempo de ejecución, aumentando la confiabilidad.
- Manejo de concurrencia → Ada tiene concurrencia nativa mediante **tasks**. Incluye mecanismos de sincronización como **rendezvous**, facilitando la programación concurrente segura.

Lenguaje JAVA

Ejercicio 6: Diga para qué fue, básicamente, creado Java ¿Qué cambios le introdujo a la Web? ¿Java es un lenguaje dependiente de la plataforma en dónde se ejecuta? ¿Por qué?

Java fue creado para desarrollar programas **portables**, que puedan ejecutarse en distintos sistemas sin modificaciones ("write once, run anywhere"). Permitió agregar **interactividad y dinamismo**, primero con applets en el navegador y luego con aplicaciones web más complejas del lado del servidor. Es **independiente de la plataforma** porque el código se ejecuta en la **JVM (Java Virtual Machine)**, que actúa como intermediaria entre el programa y el sistema operativo.

Ejercicio 7: ¿Sobre qué lenguajes está basado?

Java está basado principalmente en:

- **C y C++** → sintaxis y estructura.
- **Smalltalk** → conceptos de programación orientada a objetos.

Ejercicio 8: ¿Qué son los applets? ¿Qué son los servlets?

- **Applets**: programas Java pequeños que se ejecutaban en el navegador web. Hoy están en desuso.
- **Servlets**: programas Java que se ejecutan en el servidor, procesan solicitudes web y generan respuestas (por ejemplo, páginas dinámicas).

Lenguaje C

Ejercicio 9: ¿Cómo es la estructura de un programa escrito en C? ¿Existe anidamiento de funciones?

Un programa en C se organiza en:

- **Directivas del preprocesador** (#include, #define)
- **Declaración de variables y funciones**
- **Función principal main()**, donde comienza la ejecución
- **Funciones auxiliares** definidas por el programador

En C **no se pueden definir funciones dentro de otras funciones** (no hay anidamiento), aunque sí se pueden **llamar** entre ellas.

Ejercicio 10: Describa el manejo de expresiones que brinda el lenguaje.

C ofrece gran flexibilidad en expresiones:

- **Aritméticas:** +, -, *, /, %
- **Relacionales:** ==, !=, <, >
- **Lógicas:** &&, ||, !
- **Asignación:** =, +=, -=
- **Incremento/decremento:** ++, --
- **Condicional:** operador ternario ? :

Permite expresiones complejas combinando operadores.

Tiene **prioridad y asociatividad** de operadores.

Permite asignaciones dentro de expresiones.

Es potente, pero puede generar errores si no se usa con cuidado (menos control de tipos que otros lenguajes).

Lenguajes: Python - RUBY - PHP

Ejercicio 11: ¿Qué tipo de programas se pueden escribir con cada uno de estos lenguajes? ¿A qué paradigma responde cada uno? ¿Qué características determinan la pertenencia a cada paradigma?

Python

- **Tipo de programas:** web, automatización, ciencia de datos, IA, scripts.
- **Paradigma:** multiparadigma (imperativo, orientado a objetos, funcional).
- **Características:** tipado dinámico, uso de clases y funciones, soporta funciones como objetos.

Ruby

- **Tipo de programas:** desarrollo web (especialmente con Rails), scripts.
- **Paradigma:** orientado a objetos puro.
- **Características:** todo es objeto, uso intensivo de clases y métodos, sintaxis flexible.

PHP

- **Tipo de programas:** aplicaciones web del lado del servidor.
- **Paradigma:** principalmente imperativo y orientado a objetos.
- **Características:** se integra con HTML, ejecución en servidor, manejo de formularios y sesiones.

Ejercicio 12: Cite otras características importantes de Python, Ruby, PHP, Gobstone y Processing. Por ejemplo: tipado de datos, cómo se organizan los programas, etc.

- **Python:** tipado dinámico y fuerte**, s**intaxis simple e indentación obligatoria, programas organizados en módulos y paquetes.
- **Ruby:** tipado dinámico, todo es objeto, alta legibilidad y flexibilidad sintáctica.
- **PHP:** tipado dinámico**,** código embebido en HTML**,** amplio soporte para bases de datos.
- **Gobstone:** lenguaje educativo, imperativo, se usa para enseñar lógica de programación (tablero con piedras), sintaxis simple.
- **Processing:** orientado a gráficos y animaciones, basado en Java, usado para arte digital y visualizaciones, programas organizados en funciones como setup() y draw().

Lenguaje Javascript

Ejercicio 13: ¿A qué tipo de paradigma corresponde este lenguaje? ¿A qué tipo de Lenguaje pertenece?

JavaScript pertenece a multiparadigma (imperativo, orientado a objetos basado en prototipos y funcional). Lenguaje **interpretado**, de **alto nivel**, usado principalmente para desarrollo web (cliente y también servidor con Node.js)

Ejercicio 14: Cite otras características importantes de javascript. Tipado de datos, excepciones, variables, etc.

- **Tipado:** dinámico y débil.
- **Variables:** var, let, const (distintos alcances).
- **Funciones:** de primera clase (se pueden pasar como parámetro y retornar).
- **Objetos:** basados en prototipos (no clases tradicionales, aunque existen class).
- **Manejo de excepciones:** try, catch, finally, throw.
- **Asincronismo:** manejo con callbacks, promesas y async/await.
- **Ejecución:** principalmente en navegador, también en servidor.